

VLSI Architecture for 8-bit mini Microprocessor

Project Report

Bachelor of Technology

in

Electronics and Communication Engineering

by

Tanushka Sharma

Roll Number: 2301227

Submitted to

Dr. Rakesh Biswas



Department of Electronics and Communication Engineering

Indian Institute of Information Technology Guwahati

13th May, 2026

Contents

1. Introduction	4
2. Motivation	5
3. Theoretical Background	6
4. 4.1. Proposed New Technique.	11
4.2. Algorithm for 8-Bit Microprocessor	13
4.3. Implementation of New Technique	19
4.4. Result and Discussion	24
5. Conclusion and Future Work	26
Bibliography	27

Abstract

Reduced Instruction Set Computer (RISC) architecture improves processor performance by simplifying instruction sets, minimizing memory access, and emphasizing register-based operations. In RISC systems, computations are primarily performed within processor registers using fixed-length instructions and limited addressing modes, enabling faster decoding and execution. Techniques such as pipelining further enhance throughput by overlapping instruction stages including fetch, decode, and execute. However, the design of compact and efficient Arithmetic Logic Units (ALUs) and register files remains critical for achieving low-area and low-power processor implementations, particularly in embedded and FPGA-based systems.

This work presents the design and implementation of an 8-bit ALU and Register File using Verilog HDL as core components of a mini processor system. The ALU supports a range of arithmetic and logical operations, including addition, subtraction, multiplication, division, increment, decrement, AND, OR, and XOR, along with accurate generation of status flags such as Zero, Carry, and Auxiliary Carry. The register file is designed with multiple general-purpose registers to enable efficient data storage and transfer. The design emphasizes deterministic operation through proper initialization, controlled data flow, and reliable handling of exceptional conditions such as division by zero. A modular design methodology is adopted using standard combinational and sequential building blocks, and functional simulation verifies the correctness and robustness of the system.

For hardware validation, the design is synthesized on a Xilinx xc7a35tcpg236-1 FPGA device. The implementation demonstrates compact resource utilization, occupying 208 Look-Up Tables (1.00% of 20,800), 108 Flip-Flops (0.26% of 41,600), 10 I/O pins (9.43% of 106), and one global buffer (3.13% of 32). The total on-chip power consumption is measured at 3.164 W, with a junction temperature of 40.8 °C and a thermal margin of 44.2 °C. Although the power estimation confidence is reported as low, the results indicate that the proposed design achieves efficient performance with minimal resource overhead.

The proposed implementation demonstrates a compact, reliable, and scalable processor component design, making it suitable for low-cost and low-power embedded system applications.

Chapter 1

Introduction

Processors are the core components of modern electronic systems, enabling computation and control in devices such as computers, mobile phones, and embedded systems. They are broadly classified into general-purpose processors and dedicated processors (ASICs). General-purpose processors execute a wide variety of instructions, whereas dedicated processors are optimized for specific tasks, offering higher efficiency and lower complexity.[1]

At the hardware level, processors are built using combinational and sequential circuits formed from logic gates and memory elements like flip-flops. These are organized into key modules such as registers, Arithmetic Logic Units (ALUs), and control units. Together, these modules form the datapath and control unit, which define processor functionality.

Processor architectures have evolved from Complex Instruction Set Computing (CISC) to Reduced Instruction Set Computing (RISC). While CISC offers a large instruction set with higher complexity, RISC focuses on simpler instructions and improved performance through reduced memory access and pipelining. Due to these advantages, RISC-based designs are widely used in modern systems.

This work focuses on the design and implementation of an 8-bit ALU and Register File using Verilog HDL, ensuring reliable operation without undefined (“X”) states during simulation. The ALU supports arithmetic and logical operations such as ADD, SUB, MUL, DIV, INC, DEC, AND, OR, and XOR, along with proper generation of status flags (Zero, Carry, and Auxiliary Carry). The register file is designed with proper initialization and controlled data access to ensure stable outputs.

Existing designs include FPGA-based 8-bit and 16-bit processors using modular approaches with ALU, control unit, and register file components. Although these designs provide benefits such as modularity, hardware validation, and tool compatibility, they suffer from key limitations. These include improper register initialization leading to “X” states, incorrect flag computation, lack of division-by-zero handling, and absence of robust reset mechanisms, resulting in unreliable performance.[2][7]

To address these issues, this work implements proper initialization, accurate flag handling, safe operation under exceptional conditions, and improved coordination between modules. This ensures stable simulation and reliable processor behaviour.

Chapter 2

Motivation

Challenge of Reliable 8-bit Processor Design on FPGA:

- Designing a fully functional 8-bit processor on FPGA using Verilog HDL requires careful coordination between multiple modules — ALU, control unit, register file, memory, and I/O — each of which must operate correctly both in simulation and post-implementation.
- Existing FPGA-based 8-bit processor implementations often focus on demonstrating basic functionality but overlook critical hardware design practices, leading to designs that fail during synthesis or produce unreliable simulation results.

Limitations of Existing Methods:

- Walia et al. [1] proposed an 8-bit RISC processor on FPGA but did not address proper register initialization, resulting in undefined ('X') states at simulation startup that propagate through the datapath and corrupt output values.
- Khan et al. [2] implemented an ALU-centric 8-bit processor where carry flag detection was limited to addition only, leaving increment and other arithmetic operations without overflow detection, reducing the reliability of branch instructions that depend on the carry flag.
- Several published designs [3], [4] use blocking assignments (=) inside clocked always blocks, which violates IEEE Std 1364-2001 synthesis guidelines and introduces simulation race conditions that may not appear in behavioural simulation but cause functional failures in post-implementation testing.

Chapter 3

Theoretical Background

3.1 Microprocessor

A microprocessor is a programmable electronic device that acts as the central processing unit (CPU) of a digital system. It is fabricated on a single integrated circuit chip and is capable of performing arithmetic and logical operations, as well as controlling the overall operation of the system. A microprocessor consists of three fundamental components: the Arithmetic Logic Unit (ALU), a register array, and a control unit.[7]

The ALU performs arithmetic operations such as addition, subtraction, multiplication, and division, along with logical operations such as AND, OR, and XOR. The register array consists of a set of high-speed storage elements used to temporarily hold data, instructions, and intermediate results during execution. The control unit coordinates all operations by generating appropriate control signals to direct data flow between different components.

The working of a microprocessor follows a sequence known as the instruction cycle, which includes three main stages: Fetch, Decode, and Execute. In the fetch stage, instructions are retrieved from memory. In the decode stage, the instruction is interpreted to determine the required operation. In the execute stage, the corresponding operation is performed by the ALU, and the result is stored in registers or memory. This cycle continues until the program execution is complete.[8]

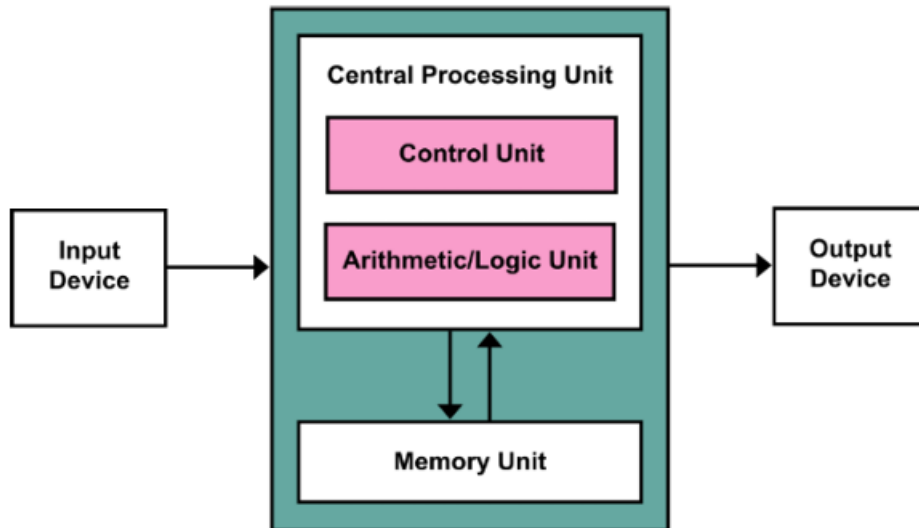


Figure 1: Von Neumann Architecture [6]

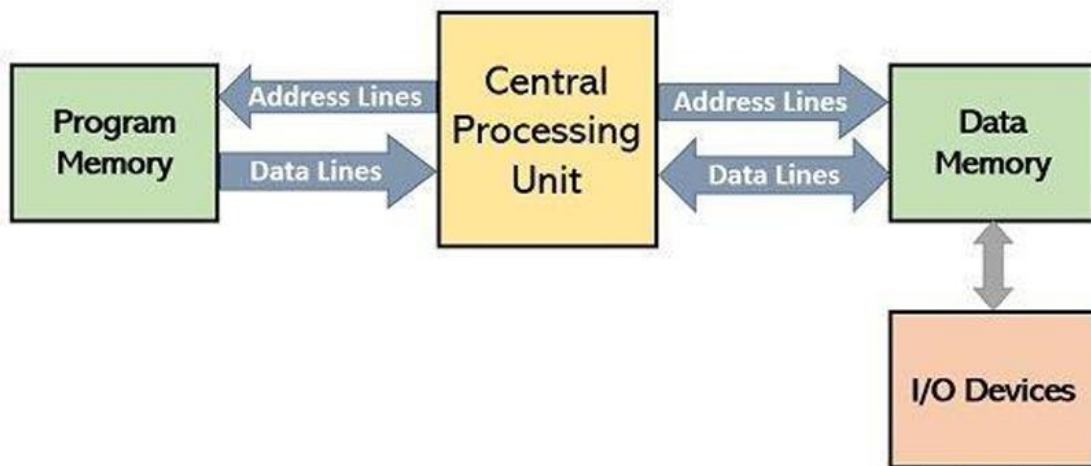


Figure 2 : Harvard Architecture [6]

3.2 Processor Architecture: CISC and RISC

Microprocessors are designed based on two primary architectures: Complex Instruction Set Computing (CISC) and Reduced Instruction Set Computing (RISC).

CISC architecture is characterized by a large and complex instruction set, multiple addressing modes, and variable instruction lengths. These features allow complex operations to be executed using fewer instructions. However, the complexity of instruction decoding and frequent memory access increases execution time and hardware complexity. The control unit in CISC processors is typically more intricate, requiring more chip area and power.

RISC architecture, on the other hand, simplifies the instruction set by using a smaller number of instructions with fixed lengths and limited addressing modes. Most operations in RISC

processors are performed using registers, and memory access is restricted to load and store instructions. This design reduces instruction complexity and allows faster execution. Additionally, RISC architectures support pipelining, where multiple instructions are processed simultaneously at different stages, significantly improving throughput.[8]

Due to these advantages, RISC processors are widely used in modern systems, especially in embedded and FPGA-based applications, where performance, power efficiency, and simplicity are critical.

3.3 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is one of the most important components of a processor. It is responsible for performing all arithmetic and logical operations required by the system. The ALU operates on binary data inputs and produces results along with status flags that indicate the outcome of operations.

Arithmetic operations include addition, subtraction, multiplication, division, increment, and decrement. Logical operations include AND, OR, XOR, and other bitwise manipulations. In addition to producing results, the ALU generates important status flags such as:

- Zero Flag (Z): Indicates whether the result of an operation is zero
- Carry Flag (C): Indicates carry-out or overflow from arithmetic operations
- Auxiliary Carry Flag (AC): Indicates carry between lower nibbles (used in BCD operations)

These flags are used by the control unit to make decisions, such as conditional branching and execution control.

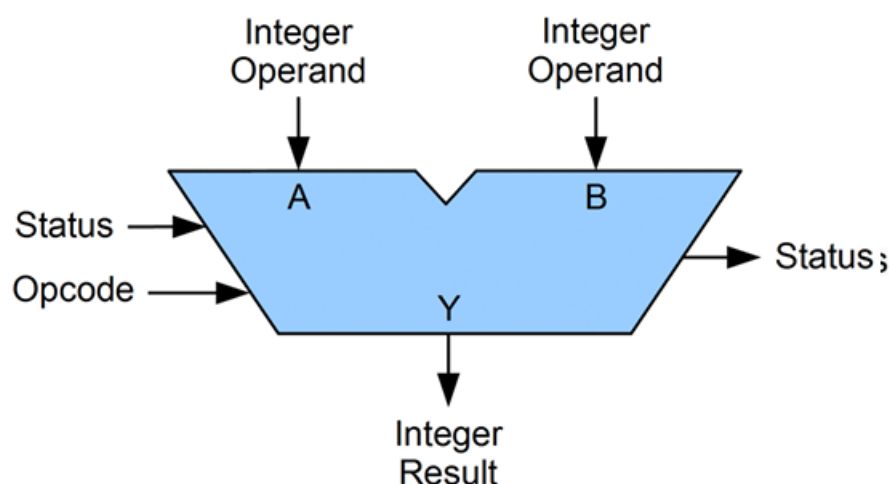


Figure 3: Arithmetic and Logic Unit (ALU) [6]

3.4 Control Unit

The Control Unit (CU) is a critical component of a microprocessor responsible for directing and coordinating the operations of all other units within the system. It acts as the brain of the processor by generating control signals that manage the flow of data between the ALU, register file, memory, and input/output devices.

The primary function of the control unit is to interpret instructions and control their execution. It does not perform any data processing itself but ensures that each component performs its assigned task at the correct time. The control unit operates based on the instruction cycle, which consists of three main stages: Fetch, Decode, and Execute.

- In the Fetch stage, the control unit generates signals to read the instruction from memory using the program counter.
- In the Decode stage, the fetched instruction is analyzed to determine the operation to be performed and the operands involved.
- In the Execute stage, appropriate control signals are generated to activate the ALU, registers, and other components to perform the required operation.

The control unit generates various control signals such as:

- Register load signals to store data in registers
- ALU control signals to select specific operations
- Memory read/write signals for data transfer
- Program counter control signals for sequential or branching execution

Control units can be broadly classified into two types: hardwired control units and microprogrammed control units. Hardwired control units use fixed logic circuits to generate control signals, resulting in faster operation but less flexibility. Microprogrammed control units use stored instructions (microinstructions) to generate control signals, offering greater flexibility at the cost of speed.[7]

In this work, the control unit is designed using a Finite State Machine (FSM) approach, where each state corresponds to a stage in the instruction cycle. The FSM transitions between states based on clock signals and instruction conditions, ensuring proper sequencing of operations.

The control unit plays a vital role in ensuring synchronized operation of all processor components. A well-designed control unit improves system reliability, prevents data hazards, and ensures correct execution of instructions, making it an essential part of processor design.

3.5 Register File

The register file is a collection of high-speed storage registers implemented using sequential logic elements such as flip-flops. It is used to store operands, intermediate results, and final outputs during program execution. Registers provide much faster access compared to main memory, thereby improving overall system performance.

In a typical processor, the register file consists of multiple general-purpose registers that can be accessed for reading and writing operations. The use of registers reduces the need for frequent memory access, which is a key feature of RISC architecture. Efficient design of the register file ensures proper data flow, reduced latency, and reliable system operation.

3.6 RISC-Based Processor Design

RISC processors are designed with the objective of achieving high performance through simplicity and efficiency. They follow a load-store architecture, where only load and store instructions access memory, while all other operations are performed on register contents. This reduces memory bottlenecks and speeds up execution.

RISC processors typically use:

- Fixed-length instructions
- Limited addressing modes
- Large number of general-purpose registers
- Single-cycle instruction execution (in ideal cases)
- Pipelining for improved throughput

These features enable faster instruction execution and efficient hardware implementation. Due to their simplicity and performance advantages, RISC processors are widely used in embedded systems and FPGA-based designs.

3.7 Verilog HDL in Processor Design

Verilog Hardware Description Language (HDL) is widely used for modeling, designing, and simulating digital systems. It allows designers to describe hardware behaviour and structure at different levels of abstraction.

In processor design, Verilog enables:

- Modular design of components such as ALU and register file
- Simulation and verification of functionality before hardware implementation
- Synthesis into FPGA hardware for real-time validation

The use of Verilog ensures flexibility, scalability, and efficient design implementation, making it suitable for developing processor components.

Chapter : 4

4.1 Proposed Methodology

The proposed methodology focuses on the design and implementation of an 8-bit processor subsystem consisting of an Arithmetic Logic Unit (ALU), Register File, and Control Unit using Verilog Hardware Description Language (HDL). The design follows a modular approach based on RISC principles to achieve simplicity, efficiency, and reliable operation suitable for FPGA implementation.

A. Design Strategy

A modular design methodology is adopted, where individual components such as the ALU, register file, and control unit are designed and verified independently before system integration. This approach improves design clarity, simplifies debugging, and enhances scalability. Verilog HDL is used to describe the hardware behaviour, enabling both simulation and synthesis for FPGA deployment.

B. ALU Implementation

The ALU is designed as a combinational logic unit capable of performing arithmetic and logical operations including addition, subtraction, multiplication, division, increment, decrement, AND, OR, and XOR. The design ensures correct computation of results along with accurate generation of status flags such as Zero (Z), Carry (C), and Auxiliary Carry (AC). Appropriate logic is incorporated to handle exceptional conditions, ensuring stable and predictable outputs.

C. Register File Implementation

The register file is implemented using sequential logic elements (flip-flops) and consists of multiple general-purpose registers. It supports synchronous read and write operations for efficient data storage and retrieval. A reset mechanism is included to initialize all registers to known values, ensuring deterministic behaviour and eliminating undefined ("X") states during simulation and operation.

D. Control Unit Design

The control unit is implemented using a Finite State Machine (FSM) to control the execution of instructions. It operates through three primary stages: Fetch, Decode, and Execute. Based on the instruction and current state, the control unit generates appropriate control signals for the ALU, register file, and data path, ensuring correct sequencing and synchronization of operations.

E. System Integration

The ALU, register file, and control unit are integrated to form a complete datapath. Proper signal connectivity is ensured to maintain correct data flow between modules. Control signals are synchronized with data operations to avoid timing issues and ensure consistent system performance.

F. Simulation and Functional Verification

The complete design is verified through simulation using standard HDL simulation tools. Various test cases are applied to validate arithmetic and logical operations, flag generation, and overall system functionality. The simulation ensures correctness and stability of the design under different operating conditions.

G. FPGA Implementation and Analysis

The verified design is synthesized and implemented on an FPGA platform. Resource utilization, including Look-Up Tables (LUTs), Flip-Flops (FFs), I/O pins, and clock resources, is analyzed to evaluate hardware efficiency. Power consumption and thermal characteristics are also examined to ensure suitability for low-power embedded applications.

H. Design Considerations

The proposed methodology emphasizes:

- Proper initialization to ensure deterministic outputs
- Accurate flag generation for all operations
- Safe handling of exceptional cases
- Efficient FPGA resource utilization
- Modular and scalable design

I. Justification of the Methodology

The chosen methodology is effective due to its combination of RISC-based design principles, modular architecture, and hardware-level implementation using Verilog HDL. It reduces system complexity, improves execution efficiency, and ensures compatibility with FPGA platforms. This approach provides a robust foundation for extending the design to more complex processor architectures in future work.

4.2 Algorithm for 8-Bit Microprocessor

This algorithm describes the operational behaviour of an 8-bit microprocessor implemented in Verilog HDL. The processor follows a classical fetch-decode-execute cycle and supports sixteen instructions including arithmetic, logical, memory access, branching, and I/O operations.

SYSTEM COMPONENTS

The processor consists of the following components:

A. Memory Units

- Instruction Memory: 256 x 8-bit ROM storing program instructions
- Data Memory: 256 x 8-bit RAM storing data for load/store operations

B. Registers

- PC (Program Counter): 8-bit register pointing to next instruction address
- IR (Instruction Register): 8-bit register holding current instruction
- ACC (Accumulator): 8-bit register for ALU operations and result storage
- MAR (Memory Address Register): 8-bit register holding memory access address
- MDR (Memory Data Register): 8-bit register holding data to/from memory
- R0-R7: Eight 8-bit general purpose registers

C. Status Flags

- ZF (Zero Flag): Set when ALU result equals zero
- CF (Carry Flag): Set when addition produces carry out of bit 7
- AF (Auxiliary Carry Flag): Set when carry occurs from bit 3 to bit 4

D. Control States

- FETCH (00): Read instruction from memory
- DECODE (01): Decode instruction and prepare operands
- EXECUTE (10): Perform the operation

MAIN ALGORITHM

Step 1: System Reset

Upon reset signal assertion:

1. Set PC = 0x00, IR = 0x00, ACC = 0x00
2. Set MAR = 0x00, MDR = 0x00
3. Clear all status flags (ZF = 0, CF = 0, AF = 0)

4. Clear all general purpose registers R0 through R7
5. Set current state = FETCH
6. Deassert all control signals

Step 2: Fetch State

1. Assert mem_read to enable instruction memory read
2. Read instruction from memory address PC into MDR
3. Assert ir_load and transfer MDR to IR
4. Assert pc_inc and increment PC by 1
5. Deassert all control signals
6. Transition to DECODE state

Step 3: Decode State

1. Extract opcode from IR bits 7 through 4
2. Extract operand from IR bits 3 through 0
3. If opcode is LOAD (0x9) or STORE (0xA):
 - a. Assert mar_load
 - b. Load operand into MAR
 - c. Deassert mar_load
4. Transition to EXECUTE state

Step 4: Execute State

Perform case analysis based on 4-bit opcode:

Case 0x0 (ADD):

- Compute 9-bit full_sum = {0, ACC} + {0, R[operand]}
- Set result = full_sum bits 7 through 0
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0
- Set CF = full_sum bit 8
- Set AF = 1 if (ACC[3:0] + R[operand][3:0]) > 15, else 0

Case 0x1 (INC):

- Compute 9-bit full_sum = {0, ACC} + 1
- Set result = full_sum bits 7 through 0
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0
- Set CF = full_sum bit 8
- Set AF = 1 if (ACC[3:0] + 1) > 15, else 0

Case 0x2 (SUB):

- Compute result = ACC - R[operand]
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0

Case 0x3 (DEC):

- Compute result = ACC - 1

- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0

Case 0x4 (MUL):

- Compute result = $\text{ACC} \times \text{R}[\text{operand}]$
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0

Case 0x5 (DIV):

- If $\text{R}[\text{operand}] \neq 0$:
 - a. Compute result = $\text{ACC} \div \text{R}[\text{operand}]$
- Else:
 - a. Set result = 0xFF (division by zero indicator)
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0

Case 0x6 (AND):

- Compute result = $\text{ACC} \& \text{R}[\text{operand}]$ (bitwise AND)
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0

Case 0x7 (OR):

- Compute result = $\text{ACC} | \text{R}[\text{operand}]$ (bitwise OR)
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0

Case 0x8 (XOR):

- Compute result = $\text{ACC} \wedge \text{R}[\text{operand}]$ (bitwise XOR)
- Store result to ACC and R[operand]
- Set ZF = 1 if result = 0, else 0

Case 0x9 (LOAD):

- Assert mem_read
- Read data from DATA_MEMORY at address MAR into MDR
- Assert mdr_load and acc_load
- Transfer MDR to ACC
- Deassert all control signals

Case 0xA (STORE):

- Assert mem_write
- Write ACC value to DATA_MEMORY at address MAR
- Deassert mem_write

Case 0xB (MOV):

- Assert reg_write
- Transfer ACC value to R[operand]
- Deassert reg_write

Case 0xC (JMP):

- Assert pc_load
- Load PC with operand address
- Deassert pc_load

Case 0xD (JZ - Jump if Zero):

- If ZF = 1:
 - a. Assert pc_load
 - b. Load PC with operand address
 - c. Deassert pc_load
- Else: Continue to next instruction (no action)

Case 0xE (JC - Jump if Carry):

- If CF = 1:
 - a. Assert pc_load
 - b. Load PC with operand address
 - c. Deassert pc_load
- Else: Continue to next instruction (no action)

Case 0xF (OUT):

- Assert out_write
- Transfer ACC value to OUTPUT_PORT
- Deassert out_write

Default:

- No operation (NOP)

Step 5: Iteration

Return to FETCH state and repeat from Step 2.

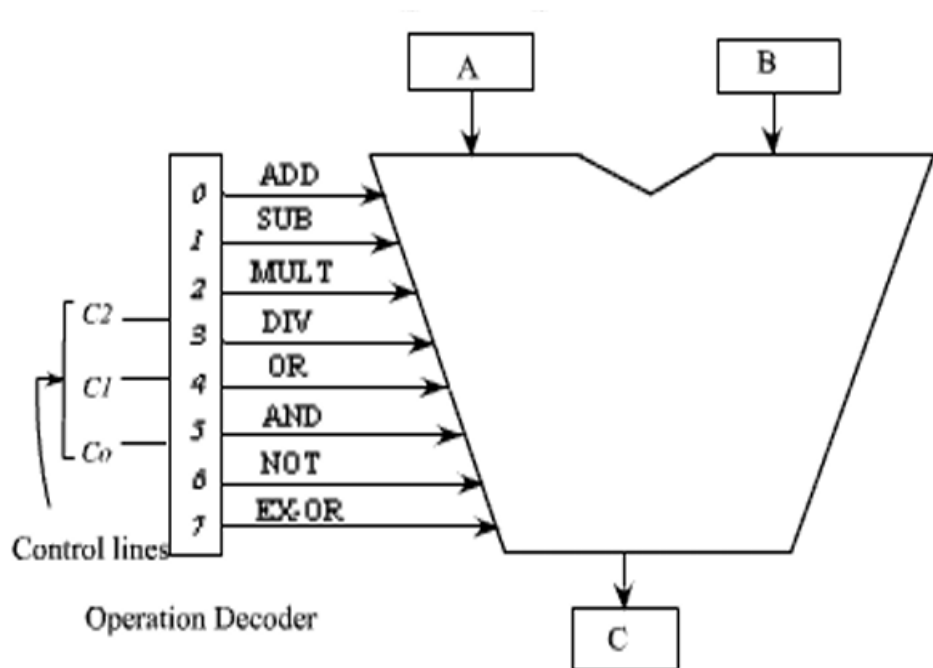


Figure 4: Arithmetic and Logic Unit Operation

IV. ALU OPERATION SUB-ALGORITHM

Input: A (8-bit), B (8-bit), opcode (4-bit)

Output: result (8-bit), carry (1-bit), aux_carry (1-bit)

Procedure:

Case opcode of:

0x0 (ADD):

$\text{full_sum} = \{0, A\} + \{0, B\}$

$\text{result} = \text{full_sum}[7:0]$

$\text{carry} = \text{full_sum}[8]$

$\text{aux_carry} = 1$ if $(A[3:0] + B[3:0]) > 15$ else 0

Return (result, carry, aux_carry)

0x1 (INC):

$\text{full_sum} = \{0, A\} + 1$

$\text{result} = \text{full_sum}[7:0]$

$\text{carry} = \text{full_sum}[8]$

$\text{aux_carry} = 1$ if $(A[3:0] + 1) > 15$ else 0

Return (result, carry, aux_carry)

0x2 (SUB):

$\text{result} = A - B$

Return (result, 0, 0)

0x3 (DEC):

$\text{result} = A - 1$

Return (result, 0, 0)

0x4 (MUL):

$\text{result} = A \times B$

Return (result, 0, 0)

0x5 (DIV):

If $B \neq 0$ then $\text{result} = A \div B$ else $\text{result} = 0xFF$

Return (result, 0, 0)

0x6 (AND):

$\text{result} = A \& B$

Return (result, 0, 0)

0x7 (OR):

$\text{result} = A | B$

Return (result, 0, 0)

0x8 (XOR):

$\text{result} = A \wedge B$

Return (result, 0, 0)

Default:

Return (0x00, 0, 0)

V. TIMING SEQUENCE**Clock Cycle N (FETCH):**

- Time T1: Apply PC address to instruction memory address bus
- Time T2: Read instruction from instruction memory
- Time T3: Load instruction into Instruction Register

- Time T4: Increment Program Counter

Clock Cycle N+1 (DECODE):

- Time T1: Extract opcode and operand from Instruction Register
- Time T2: For memory operations, load Memory Address Register
- Time T3: Transition to EXECUTE state

Clock Cycle N+2 (EXECUTE):

For ALU Operations:

- Time T1: Select ALU inputs (ACC and R[operand])
- Time T2: Perform ALU computation
- Time T3: Write result to ACC and register file
- Time T4: Update status flags

For Memory Operations:

- Time T1: Apply address to data memory
- Time T2: Read or write data
- Time T3: For LOAD, transfer data to ACC

For Jump Operations:

- Time T1: Check condition for conditional jumps
- Time T2: Load PC with new address if condition true

For Output Operation:

- Time T1: Latch ACC value to output port

Clock Cycle N+3:

- Return to FETCH state with new PC value

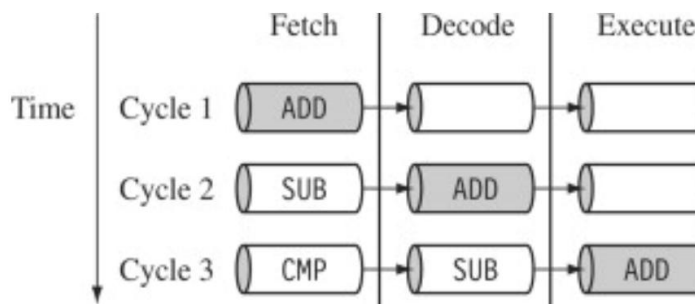


Figure 5: Pipelining Of 3 stage Instructions cycle

4.3 Implementation

Hardware Implementation Results:

a.) FPGA Implementation: The proposed 8-bit microprocessor is implemented and synthesised on the Xilinx Artix-7 (xc7a35tcbg236-1) device featuring 208 LUTs, 108 flip-flops, 10 I/O pins, and 1 BUFG.

- The on-board 100 MHz oscillator provides the system clock, which is routed through a global clock buffer (BUFG) to all sequential modules — confirmed by the single BUFG entry in the post-implementation utilisation report.
- The Basys 3 provides 16 slide switches, 16 LEDs, 4 seven-segment displays, and 5 pushbuttons, all of which are used for input stimulus and output observation without any external hardware.

Observing Output on the Basys 3 Board

Output Mapping:

- The 8-bit port out signal from the microprocessor top module is mapped directly to LEDs LD7–LD0 on the board. Each LED corresponds to one bit of the output register — LD7 is the most significant bit and LD0 is the least significant bit. When the processor executes the OUT instruction and writes the accumulator (e.g. 0x1E = 30 decimal = 00011110 binary), LEDs LD4, LD3, LD2, LD1 illuminate and the remaining LEDs stay off.
- The reset pushbutton is mapped to the reset input. Pressing it drives reset high, clears the program counter, accumulator, flags, and output register to 0x00, and extinguishes all LEDs. Releasing it allows the processor to begin fetching from address 0x00 and re-execute the program.
- The on-board clock (W5, 100 MHz) is used directly. No clock divider is needed for functional demonstration; the processor operates comfortably within Artix-7 timing closure at this frequency given its combinational depth.

b.) Submodule Descriptions

1. Program Counter (`program_counter.v`)

- An 8-bit register that holds the address of the next instruction to be fetched from instruction memory.

- On every clock edge: if `rst` is asserted it resets to `0x00`; if `load` is high it captures `jump_addr` (from the instruction operand) for `JMP/JZ/JC` instructions; if `inc` is high it increments by 1 for normal sequential execution.
- Outputs `pc_out` which feeds directly into the instruction memory address port.

2. Instruction Memory (`instruction_memory.v`)

- A 256×8 -bit synchronous ROM that stores the program to be executed.
- The program is hardcoded using `mem[i] = 8'hXX` statements in an initial block so that it is visible to the Vivado synthesiser and is mapped to distributed LUT-RAM on the Artix-7.
- On every rising clock edge it presents `mem[pc_out]` on `instr_word`, which is the raw 8-bit instruction byte passed to the instruction register.

3. Instruction Register (`instruction_register.v`)

- An 8-bit latch that holds the instruction fetched from memory stable throughout the decode and execute stages of the pipeline.
- When `ir_load` is asserted (during the FETCH stage by the control unit), it captures `instr_word` on the rising clock edge.
- Its output `ir_out[7:4]` carries the 4-bit opcode to the control unit, and `ir_out[3:0]` carries the 4-bit operand (register select or memory address) to the register file, MAR, MDR, and program counter.

4. Control Unit (`control_unit.v`)

- A 2-bit three-state FSM with states `FETCH (2'b00)`, `DECODE (2'b01)`, and `EXECUTE (2'b10)` that advances sequentially on every clock edge and wraps back to `FETCH` after `EXECUTE`.
- During `FETCH`: asserts `mem_read`, `ir_load`, and `pc_inc` simultaneously to fetch the instruction, latch it, and advance the PC.
- During `DECODE`: asserts `mar_load` to latch the instruction operand into the MAR, preparing the memory address for load/store operations.
- During `EXECUTE`: decodes `ir_out[7:4]` (the opcode) and asserts the appropriate combination of `alu_op`, `acc_load`, `reg_write`, `mem_read`, `mem_write`, `pc_load`, and `out_write` based on the instruction type.
- For conditional branches (`JZ`, `JC`), the FSM reads `zero_flag` and `carry_flag` from the flags register to decide between `pc_load` (branch taken) and `pc_inc` (branch not taken).

5. Register File (`register.v`)

- Eight general-purpose 8-bit registers (`R0–R7`) implemented as a register array `pin[0:7]`.

- sel_in[2:0] selects the destination register for write operations; sel_out[2:0] selects the source register for read operations.
- When enable is high on a rising clock edge, data_in is written to pin[sel_in].
- When output_port is high, pin[sel_out] is driven onto data_out; when low, data_out is driven to 8'bz for shared-bus compatibility.
- In the processor, data_in is connected to alu_result so that every ALU output can be written back to any register.

6. ALU (alu.v)

- A purely combinational module that takes 8-bit operands A (from accumulator) and B (from register file) and performs the operation selected by alu_op[3:0].
- Supports nine operations: ADD (0000), INC (0001), SUB (0010), DEC (0011), MUL (0100), DIV (0101), AND (0110), OR (0111), XOR (1000).
- Uses a 9-bit internal register full_sum for ADD and INC to correctly capture the carry-out from bit 7 into bit 8.
- Generates three combinational flag outputs: zero_flag (result == 0), carry_flag (full_sum[8]), and auxiliary_carry_flag (lower nibble carry: A[3:0] + B[3:0] > 4'hF).

7. Accumulator (accumulator.v)

- A dedicated 8-bit register that holds the primary ALU result between instructions, acting as the implicit source and destination for all arithmetic and logical operations.
- Loaded from alu_result when acc_load is asserted by the control unit during the EXECUTE stage.
- Its output acc_out feeds both the A input of the ALU and the data input of the output port, the data memory write port, and the MDR.

8. Flags Register (flags_register.v)

- Three single-bit registers — zero_flag, carry_flag, and auxiliary_carry_flag — that capture the status of the most recent ALU operation.
- Updated from the ALU's combinational flag outputs when update (tied to acc_load) is asserted, ensuring flags always reflect the result currently loaded into the accumulator.
- Read by the control unit during the EXECUTE stage of JZ and JC instructions to determine whether the branch is taken.

9. MAR and MDR (mar_mdr.v)

- **Memory Address Register (MAR):** An 8-bit register that latches ir_out[7:0] (the instruction operand) when mar_load is asserted during the DECODE stage. Its output

mar_addr is presented to the data memory address port throughout the EXECUTE stage, ensuring the address is stable when mem_read or mem_write fires.

- **Memory Data Register (MDR):** An 8-bit register that latches dmem_out (the data memory read output) when mdr_load is asserted. For LOAD instructions, the latched value is forwarded to the accumulator via mdr_data.

10. Data Memory (data_mem.v)

- A 256×8 -bit synchronous RAM that stores runtime data values.
- On a rising clock edge: if mem_write is high, acc_out is written to ram[mar_addr]; if mem_read is high, ram[mar_addr] is presented on data_out.
- Initialised to all zeros in an initial block so that simulation reads before any write return 0x00 rather than X.

11. Output Port (output_port.v)

- An 8-bit register that latches acc_out when out_write is asserted by the control unit during the EXECUTE stage of the OUT instruction.
- Its output port_out is the top-level output of microprocessor_top and is routed directly to LEDs LD7–LD0 on the Basys 3 via the XDC constraints file.
- Cleared to 0x00 on reset, which extinguishes all LEDs immediately when BTNC is pressed.

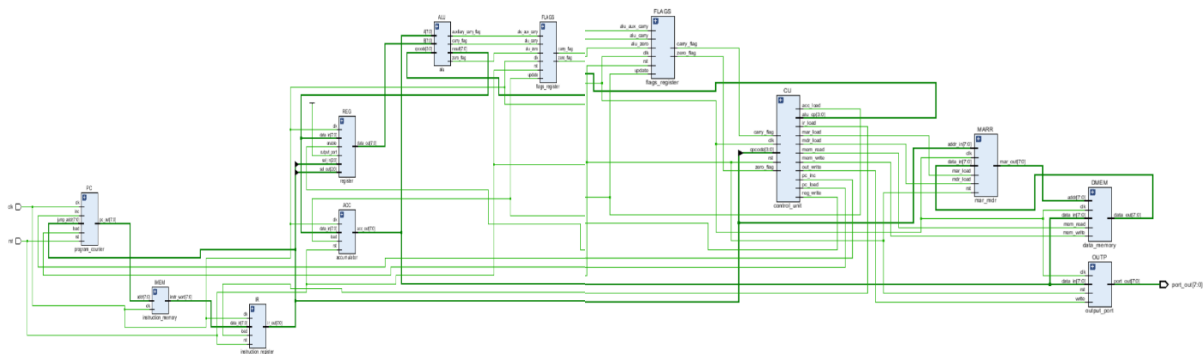


Figure 6 : Schematic diagram of 8 bit Microprocessor

DATA FLOW DIAGRAM OF 8-BIT MICROPROCESSOR

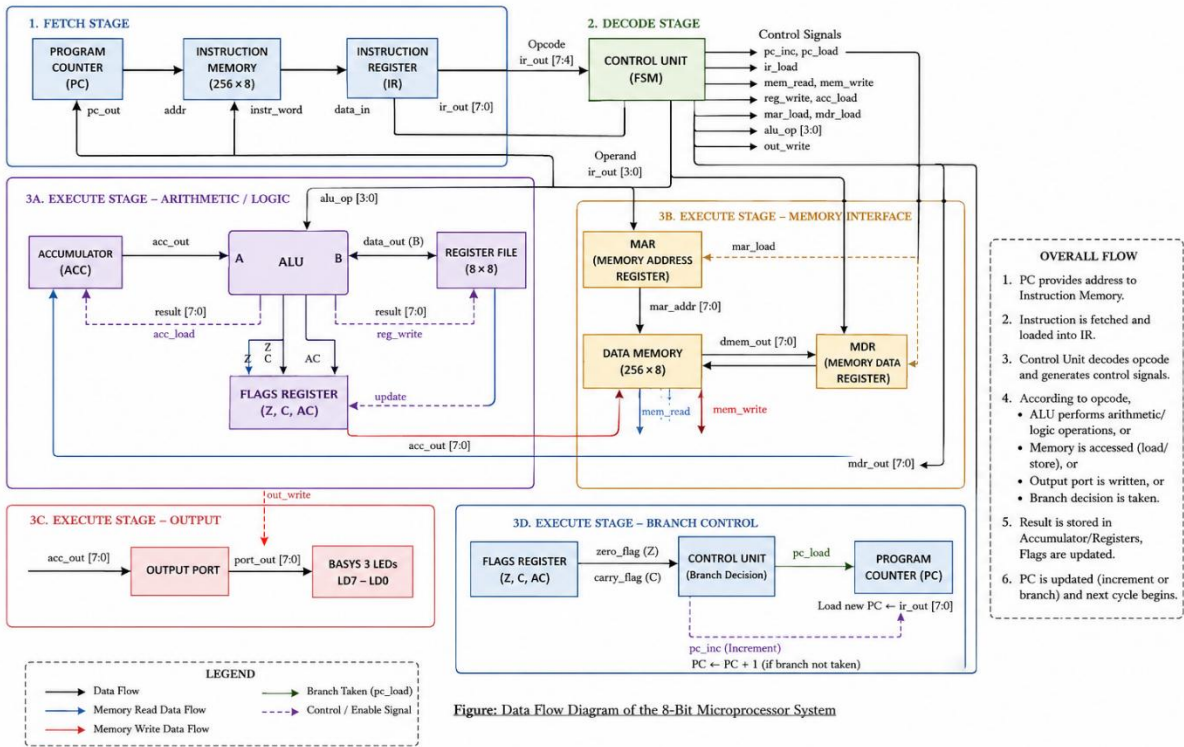


Figure: Data Flow Diagram of the 8-Bit Microprocessor System

Figure 7: Data Flow of 8-bit Microprocessor System [6]

4.4 Results and Discussion

Hardware Implementation Results:

a.) FPGA Implementation: The proposed 8-bit microprocessor is implemented and synthesised on the Xilinx Artix-7 (xc7a35tcp236-1) device featuring 208 LUTs, 108 flip-flops, 10 I/O pins, and 1 BUFG.

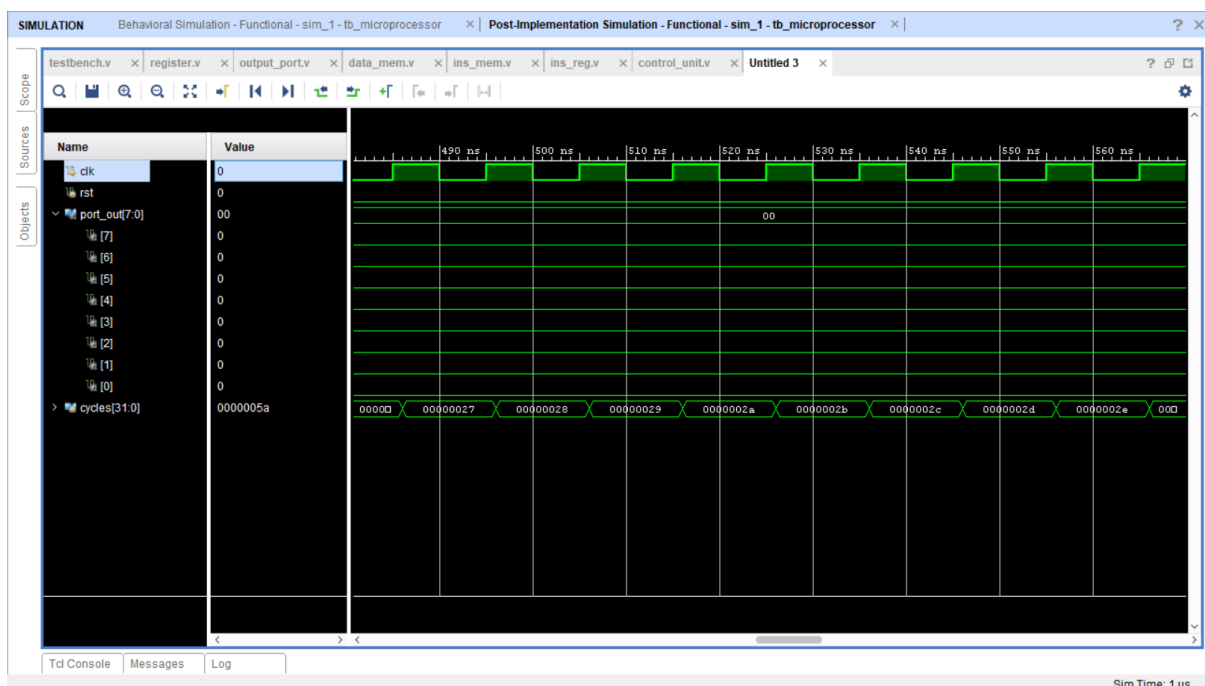


Figure 8: Post-implementation functional simulation waveform of the 8-bit microprocessor showing clock signal, reset behaviour, cycle progression, and stable output port values during execution.

Comparison of Existing and Proposed Design :

Parameter	Existing Designs	Proposed Design
Register Initialization	Not defined, causes 'X' states	Proper reset, no undefined states
ALU & Flags	Limited or inaccurate	Accurate operations and flag generation

Parameter	Existing Designs	Proposed Design
Error Handling	No divide-by-zero handling	Safe and defined behaviour
Control Unit	Complex / poorly synchronized	FSM-based (Fetch–Decode–Execute)
Simulation	Unstable, ‘X’ propagation	Stable and deterministic
FPGA Usage	Moderate	Low and efficient
Design Approach	Less modular	Highly modular and scalable

Chapter:5

Conclusion and Future Work

In this work, an 8-bit RISC-based microprocessor was successfully designed and implemented using Verilog HDL. The processor architecture includes key components such as the Arithmetic Logic Unit (ALU), Control Unit, Register File, Program Counter, and Memory modules, organized using a modular design approach. The ALU supports multiple arithmetic and logical operations, while the control unit follows a structured Fetch–Decode–Execute cycle ensuring correct instruction execution.

The functional and timing simulations demonstrate correct processor behaviour, validating all instruction types including arithmetic, logical, data transfer, and control operations. The system successfully executes the test program, producing accurate results at the output port. Proper initialization and controlled data flow ensure stable simulation without undefined (“X”) states, improving reliability compared to existing designs.

The FPGA implementation on the Xilinx (xc7s50csga324-1) device confirms that the design is compact and hardware-efficient, utilizing only 208 LUTs (1.00%), 108 flip-flops (0.26%), 10 I/O pins (9.43%), and 1 BUFG (3.13%). The design operates with a total on-chip power consumption of 3.164 W and maintains safe thermal performance, demonstrating suitability for low-area and embedded applications.

Overall, the proposed microprocessor provides a clear understanding of processor design principles and demonstrates an efficient, reliable, and scalable hardware implementation using simplified RISC architecture.

For future work, the designed microprocessor can be extended by integrating additional peripherals such as a UART module for serial communication. Further enhancements may include pipelining, interrupt handling, and support for a more complex instruction set to improve performance and functionality.

Bibliography

- [1] R. Walia and H. Singh, "Design and implementation of 8-bit RISC processor using Verilog HDL," *International Journal of Engineering Research and Technology*, vol. 6, no. 5, 2017.
- [2] M. A. Khan et al., "FPGA implementation of an 8-bit ALU and processor core using Verilog," *Proc. IEEE Int. Conf. on Computing and Communication*, 2018.
- [3] S. Mohan and R. Prasad, "Modular design of an 8-bit processor on Xilinx FPGA," *Int. Journal of VLSI Design and Communication Systems*, vol. 4, no. 3, 2013.
- [4] P. Sharma and A. Gupta, "Synthesis and simulation of a simple 8-bit CPU using Verilog HDL on Artix-7," *Proc. Int. Conf. on Signal Processing and VLSI Design*, 2019.
- [5] V. Naresh and B. Suresh, "Implementation of 8-bit microprocessor using Verilog on Xilinx Spartan-6," *Int. Journal of Advanced Research in Electronics*, vol. 2, no. 1, 2015.
- [6] Image source: Canva AI, "AI-generated image," Canva, 2026.
- [7] T. Teja, "*B.E. ECE Project Report (Batch No. 67)*", Sathyabama Institute of Science and Technology, 2023.
- [8] D. Zucchini, "*Microprocessor Design Learning*," Missouri University of Science and Technology.